

Dharani Nikesh. N.S
192424201

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

SA
41/50
82%
\$

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.
2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.

ANSWER:

Aim

To develop a Python program that generates a star topology for a given number of hosts connected to a central switch and displays the corresponding Cat6 UTP cable lengths.

Algorithm

1. Read the number of hosts from the user.
2. Create a central switch named SWITCH-1.
3. Connect every host directly to the central switch.
4. Generate a Cat6 cable length for each connection.
5. Display the host, switch, and cable length.
6. Verify that every host has a connection to the switch.

Program

```
import random

def create_star_topology():
    n = int(input("Enter number of hosts: "))

    switch = "SWITCH-1"

    print("\n===== STAR TOPOLOGY =====")
    print("                " + switch)

    for i in range(1, n + 1):
        host = f"Host-{i}"
        length = random.randint(5, 100)

        print(f"                |")
        print(f"            {host} ---- Cat6 UTP ({length} m)")

    print("\n===== CONNECTION DETAILS =====")

    for i in range(1, n + 1):
```

```

host = f"Host-{i}"
length = random.randint(5, 100)
print(f"{host} ---> {switch} ---> {length} m")

print("\nAll hosts are connected through the central switch.")

create_star_topology()

```

Sample Output

```

===== NETWORK / CONFIGURATION DOCUMENT =====
Enter number of hosts: 5

===== STAR TOPOLOGY =====
                SWITCH-1
                |
Host-1  ---- Cat6 UTP (41 m)
                |
Host-2  ---- Cat6 UTP (73 m)
                |
Host-3  ---- Cat6 UTP (15 m)
                |
Host-4  ---- Cat6 UTP (17 m)
                |
Host-5  ---- Cat6 UTP (67 m)

===== CONNECTION DETAILS =====
Host-1  ----> SWITCH-1  ----> 50 m
Host-2  ----> SWITCH-1  ----> 38 m
Host-3  ----> SWITCH-1  ----> 59 m
Host-4  ----> SWITCH-1  ----> 59 m
Host-5  ----> SWITCH-1  ----> 35 m

All hosts are connected through the central switch.

```

Result

Thus, a star topology was successfully generated using Python, and all hosts were connected to the central switch.

2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

ANSWER:

Aim

To develop a Python function `validate_segment(length_m)` to verify whether a UTP cable segment satisfies the maximum Ethernet cable length of 100 meters.

Algorithm

1. Define the function `validate_segment(length_m)`.
2. Check whether the length is between 0 and 100 meters.
3. Return `True` for a valid length.
4. For a length greater than 100 meters, return `False` with an error message.
5. Test the function with at least three values.

Program

```
def validate_segment(length_m):
    if length_m < 0:
        return False, "Error: Cable length cannot be negative."
    elif length_m <= 100:
        return True, "Valid: Cable length is within the 100 m limit."
    else:
        return False, "Invalid: Cable length exceeds the 100 m limit."

# Test cases
test_cases = [50, 100, 120]

for length in test_cases:
    result, message = validate_segment(length)
    print(f"Length = {length} m")
    print(f"Result = {result}")
    print(message)
    print()
```

Sample Output

```
----- RESTART: /usr/bin/python3.9/documen
Length = 50 m
Result = True
Valid: Cable length is within the 100 m limit.

Length = 100 m
Result = True
Valid: Cable length is within the 100 m limit.

Length = 120 m
Result = False
```

Interpretation

Cable Length Result		Interpretation
50 m	True	Valid
100 m	True	Valid maximum length
120 m	False	Exceeds permissible limit

Result

The program successfully validates UTP cable lengths and identifies cable segments exceeding the 100-meter limit.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

ANSWER:

Aim

To develop a Python program that simulates CSMA/CD operation among four hosts and records transmission attempts, collisions, and random back-off intervals in `csma_log.txt`.

Algorithm

1. Create four hosts.
2. Select a random host for transmission.
3. Check whether another host is transmitting.
4. If multiple hosts transmit simultaneously, detect a collision.
5. Generate a random back-off interval.
6. Wait for the back-off period.
7. Retry transmission.
8. Record all events in `csma_log.txt`.
9. Continue until all hosts successfully transmit.

Program

```
import random
import time

hosts = ["Host-1", "Host-2", "Host-3", "Host-4"]

log_file = "csma_log.txt"

with open(log_file, "w") as log:

    def write_log(message):
```

```

        print(message)
        log.write(message + "\n")

successful = set()
attempts = 0

while len(successful) < len(hosts):
    attempts += 1

    # Select one or more hosts randomly
    transmitting = random.sample(
        hosts,
        random.randint(1, min(2, len(hosts)))
    )

    write_log(
        f"Transmission Attempt {attempts}: "
        f"{', '.join(transmitting)}"
    )

    if len(transmitting) > 1:
        write_log("COLLISION DETECTED!")

        for host in transmitting:
            backoff = random.randint(1, 5)

            write_log(
                f"{host} Back-off interval = {backoff} units"
            )

            time.sleep(0.5)

    else:
        host = transmitting[0]

        if host not in successful:
            write_log(
                f"{host}: Frame transmitted successfully."
            )
            successful.add(host)

        write_log("-" * 40)

print("\nSimulation completed.")
print("Log saved in csma_log.txt")

```

Sample Output

```

-----
Transmission Attempt 1: Host-4
Host-4: Frame transmitted successfully.
-----
Transmission Attempt 2: Host-4
-----
Transmission Attempt 3: Host-2
Host-2: Frame transmitted successfully.
-----
Transmission Attempt 4: Host-1, Host-2
COLLISION DETECTED!
Host-1 Back-off interval = 3 units
Host-2 Back-off interval = 4 units
-----
Transmission Attempt 5: Host-3, Host-1
COLLISION DETECTED!
Host-3 Back-off interval = 3 units
Host-1 Back-off interval = 4 units
-----
Transmission Attempt 6: Host-1
Host-1: Frame transmitted successfully.
-----
Transmission Attempt 7: Host-3, Host-4
COLLISION DETECTED!
Host-3 Back-off interval = 4 units
Host-4 Back-off interval = 4 units
-----
Transmission Attempt 8: Host-2, Host-3
COLLISION DETECTED!
Host-2 Back-off interval = 4 units
Host-3 Back-off interval = 2 units
-----
Transmission Attempt 9: Host-1, Host-4
COLLISION DETECTED!
Host-1 Back-off interval = 5 units
Host-4 Back-off interval = 3 units
-----
Transmission Attempt 10: Host-3
Host-3: Frame transmitted successfully.
-----

Simulation completed.
Log saved in csma_log.txt

```

Result

The CSMA/CD protocol was successfully simulated, and transmission attempts, collisions, and back-off intervals were recorded in `csma_log.txt`.

4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

ANSWER:

Aim

To develop a Python socket-based client-server application that simulates communication between hosts through a central switch and demonstrates MAC address learning and frame forwarding.

Algorithm

Server:

1. Create a TCP socket.
2. Wait for a client connection.
3. Receive a frame containing source and destination MAC addresses.
4. Learn the source MAC address.
5. Store the source MAC and port in the MAC table.
6. Forward the frame to the appropriate port.
7. Display the updated MAC table.

Client:

1. Create a socket.
2. Connect to the server.
3. Send Ethernet-like frame information.
4. Receive the forwarding response.

Server Program

```
import socket

HOST = "127.0.0.1"
PORT = 5000

mac_table = {}

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)

print("Switch server waiting for connection...")

connection, address = server.accept()
print("Client connected:", address)

while True:
    data = connection.recv(1024).decode()

    if not data:
        break

    if data.lower() == "exit":
        break

    # Format: SOURCE_MAC, DESTINATION_MAC, PORT
    source, destination, port = data.split(",")
```



```

# MAC learning
mac_table[source] = port

print("\nFrame Received")
print("Source MAC:", source)
print("Destination MAC:", destination)

# Frame forwarding
if destination in mac_table:
    forward_port = mac_table[destination]
    message = f"Frame forwarded to port {forward_port}"
else:
    message = "Destination unknown - Frame broadcast"

connection.send(message.encode())

print("\nUpdated MAC Address Table")
print("-----")

for mac, p in mac_table.items():
    print(f"{mac} -> Port {p}")

connection.close()
server.close()

```

Client Program

```

import socket

HOST = "127.0.0.1"
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

frames = [
    "AA:AA:AA:AA:AA:01,BB:BB:BB:BB:BB:02,1",
    "BB:BB:BB:BB:BB:02,AA:AA:AA:AA:AA:01,2"
]

for frame in frames:
    client.send(frame.encode())

    response = client.recv(1024).decode()
    print("Switch:", response)

client.send("exit".encode())
client.close()

```

Sample Output

```

Client connected: ('127.0.0.1', 53520)

Frame Received
Source MAC: AA:AA:AA:AA:AA:01
Destination MAC: BB:BB:BB:BB:BB:02

Updated MAC Address Table
-----
AA:AA:AA:AA:AA:01 -> Port 1
Switch: Destination unknown - Frame broadcast

Frame Received
Source MAC: BB:BB:BB:BB:BB:02
Destination MAC: AA:AA:AA:AA:AA:01

Updated MAC Address Table
-----
AA:AA:AA:AA:AA:01 -> Port 1
BB:BB:BB:BB:BB:02 -> Port 2
Switch: Frame forwarded to port 1
Client communication completed.

```

Result

The client-server application successfully demonstrated MAC address learning and frame forwarding using Python sockets.

5. Design and implement a Python class named `StarSwitch` that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

ANSWER:

Aim

To design a Python class `StarSwitch` that models basic Ethernet switch functionality including adding ports, removing ports, broadcasting frames, and maintaining broadcast logs.

Algorithm

1. Create the `StarSwitch` class.
2. Maintain a list of active hosts.
3. Use `add_port()` to connect hosts.
4. Use `remove_port()` to disconnect hosts.
5. Use `broadcast()` to send a frame to all hosts except the source.
6. Record each broadcast operation.
7. Display the received hosts.

Program

```
class StarSwitch:

    def __init__(self):
        self.active_ports = []
        self.broadcast_log = []

    def add_port(self, host_id):
        if host_id not in self.active_ports:
            self.active_ports.append(host_id)
            print(f"{host_id} connected to switch.")

    def remove_port(self, host_id):
        if host_id in self.active_ports:
            self.active_ports.remove(host_id)
            print(f"{host_id} disconnected from switch.")

    def broadcast(self, source, frame):
        receivers = [
            host for host in self.active_ports
            if host != source
        ]

        log = (
            f"Source: {source} | "
            f"Frame: {frame} | "
            f"Receivers: {receivers}"
        )

        self.broadcast_log.append(log)

        print("\nBroadcast Operation")
        print("-----")
        print("Source:", source)
        print("Frame:", frame)
        print("Destination Hosts:", receivers)

# Create switch
switch = StarSwitch()

# Add hosts
switch.add_port("Host-1")
switch.add_port("Host-2")
switch.add_port("Host-3")
switch.add_port("Host-4")
```

```

# Broadcast frames
switch.broadcast("Host-1", "Hello Network")
switch.broadcast("Host-3", "Data Packet")

# Remove a host
switch.remove_port("Host-4")

# Another broadcast
switch.broadcast("Host-2", "Network Update")

print("\nBroadcast Log")
print("-----")

for entry in switch.broadcast_log:
    print(entry)

```

Sample Output

```

-----
Host-1 connected to switch.
Host-2 connected to switch.
Host-3 connected to switch.
Host-4 connected to switch.

Broadcast Operation
-----
Source: Host-1
Frame: Hello Network
Destination Hosts: ['Host-2', 'Host-3', 'Host-4']

Broadcast Operation
-----
Source: Host-3
Frame: Data Packet
Destination Hosts: ['Host-1', 'Host-2', 'Host-4']
Host-4 disconnected from switch.

Broadcast Operation
-----
Source: Host-2
Frame: Network Update
Destination Hosts: ['Host-1', 'Host-3']

Broadcast Log
-----
Source: Host-1 | Frame: Hello Network | Receivers: ['Host-2', 'Host-3', 'Host-4']
Source: Host-3 | Frame: Data Packet | Receivers: ['Host-1', 'Host-2', 'Host-4']
Source: Host-2 | Frame: Network Update | Receivers: ['Host-1', 'Host-3']
|

```

Result

The `StarSwitch` class was successfully implemented to manage active ports and simulate Ethernet broadcasting in a star topology.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
A Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed 1.5	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
B Algorithm Design	20%	Efficient, optimal, and well-structured algorithm 2	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
C Code Quality	15%	Clean, well-organized, readable, and properly formatted code 1.5	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
D Functionality	20%	Code works perfectly for all test cases 2	Works for most test cases	Works for limited cases	Does not run or fails major cases
E Error Handling	10%	Handles all edge cases and errors effectively 1	Handles some edge cases	Limited error handling	No error handling
F Efficiency	10%	Highly optimized in time and space complexity 1	Reasonably efficient	Some inefficiencies	Highly inefficient
G Documentation & Comments	5%	Clear and meaningful comments and documentation 0.5	Adequate comments	Few or unclear comments	No comments
H Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs 0.5	Tested with some cases	Minimal testing	No testing done

	A	B	C	D	E	F	G	H	Total
Q1	1.5	1.5	1	1.5	1	1	0.5	0.5	8.5
Q2	1	1	1	1	1	0.5	0.5	0.5	6.5
Q3	1	2	1.5	2	1	0.5	0.5	0.5	9
Q4	1.5	2	1	2	1	1	0.5	0.5	9.5
Q5	1	1.5	1.5	1	0.5	1	0.5	0.5	7.5